Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 171 (2020) 800–809

Procedia Computer Science

www.elsevier.com/locate/procedia

Third International Conference on Computing and Network Communications (CoCoNet'19)

A Low Latency Montgomery Modular Exponentiation

Venkata Reddy K[a,*], Simranjeet Singh C[a], Vivian Desalphine[a], David Selvakumar[a]

[a]*Secure Hardware and VLSI Design (SHVD) Group, Centre for Developement of Advanced Computing (C-DAC), Bangalore, India*

Abstract

RSA (Rivest-Shamir-Adleman) is a widely known and popular public-key cryptosystem used in many security applications. The main mathematical function in RSA is Modular Exponentiation, which is demanding in terms of speed and area. Due to repetitive Modular Multiplications involved in Exponentiation function, it becomes time-consuming, especially for large operands. To accelerate the RSA Encryption/Decryption operation, a reduction in number of Modular Multiplications as well as reduction in clock-cycles to perform one Modular Multiplication operation is required. The main objective of this paper is to develop high-speed, area-efficient hardware Montgomery Modular Exponentiation architecture for RSA Encryption/Decryption operations by using Lookup-Table (LUT) based high-radix Montgomery Modular Multiplier. In this paper, we present a hardware architecture designed to implement Right-to-Left (R-L) Montgomery Modular Exponentiation using a lookup-table based Radix-16 Modular Multiplication with Montgomery reduction. The proposed architecture provides tremendous improvement by reducing the number of clock cycles for one Modular Multiplication from n (operand bit-length) to n/4 cycles. With our implementation results, for Montgomery Modular Exponentiation with Radix-16 lookup-table based Montgomery Multiplier architecture, we are able to achieve computational delay of 6.35ms, 43.16ms, 320.4ms for computing 1024/ 2048/ 4096 bits of Modular Exponentiation, respectively. We were able to achieve significant improvement in terms of time and Time-Area (TA).

© 2020 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the Third International Conference on Computing and Network Communications (CoCoNet'19).

Keywords: public-key cryptography; high-radix implementation; Montgomery modular arithmetic; RSA; CSA; FPGA DSP48

1. Introduction

The increased use of computer and communication systems in industry has increased the risk of theft of proprietary information. To achieve reliable security, many different cryptographic algorithms have been proposed and tested for their security strength.

* Corresponding author. Tel.: +91-80-25093423; fax: +91-80-25247724.
E-mail address: venkatark@cdac.in

1877-0509 © 2020 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the Third International Conference on Computing and Network Communications (CoCoNet'19).
10.1016/j.procs.2020.04.087

RSA (Rivest, Shamir, and Adleman) algorithm is one of the most important public-key cryptosystem used for high secure transmission of data [1]. The main function of this algorithm is Modular Exponentiation for message Encryption/Decryption operations.

The Modular Exponentiation is employed either to encrypt a message 'M' by using a public key {e, N} or decrypt (recover original message) by using a private key {d, N}.

$$C = M^e mod\ N\ and\ M = C^d mod\ N$$

Where 'N' is the modulus of 'n' bits.

Software implementation of RSA Encryption/Decryption is far too slow for real-time applications [2]. Therefore, to attain high-speed RSA performance, dedicated and optimized hardware implementation is essential [3].

The remaining of this paper is structured as follows: in Section 2 we discuss related work. Section 3, we outline the Montgomery Modular Multiplication. Section 4 presents the hardware architecture development for computing Radix-16 Montgomery Modular Multiplication. Section 5 describes the usage of binary method used in Exponentiation implementation and Montgomery Exponentiation hardware implementation with proposed Radix-16 Modular Multiplier. Section 6 summarizes the hardware implementation reports of the proposed architecture on Virtex-6 FPGA device. Finally, we present the conclusion in Section 7.

2. Related Work

The complexity associated with computing Modular Exponentiation is discussed in [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14], where increase in bit-size of the operands leads to lower performance. This has resulted in ways to improve the performance by reducing computational time and complexity of calculating Modular Exponentiation. In [4], the authors discuss R-L Exponentiation implementations with Montgomery Modular Multiplier. In Montgomery Modular Multiplier design [4], a MUX (multiplexer) was incorporated to perform four-to-one selection according to operands instead of two multiplications and one addition in the product loop. In [5], time improvement is achieved by using Radix-4 employing Booth's technique. In [5], it is concluded that Radix-4 is the best selection for implementation of Modular Multiplication. In our proposed implementation, we were able to achieve Radix-16 Montgomery Modular Multiplier using Lookup-Table (LUT) approach instead of using Booth's scheme. Also, the area of our proposed multiplier doesn't increase linearly with radix increment as stated in [5]. In [6, 7], the authors implemented R-L Exponentiation with Radix-2 Montgomery Modular Multiplier. The work presented in this paper presents a Radix-16 Montgomery Multiplier to get very high-performance in comparison to implementations in [6, 7]. In [8, 9, 10, 11], the authors implemented high-radix Montgomery Multiplier by using Booth's encoding/recoding schemes. This requires pre-computations and extra encoding/decoding logic for multiplication. In our Montgomery Modular Multiplier implementation, no pre-computation is required. In [19], high-radix Modular Exponentiation is implemented using Left-to-Right binary method, which doesn't allow parallel multiplications; hence, to compute one Modular Exponentiation it always takes more than n multiplications delay (for any radix exponentiation). Our proposed hardware design uses Right-to-Left binary method, which allows parallel execution of modular operations 'squares and multiplications'. This takes at most n multiplications delay to compute one Modular Exponentiation.

The work presented in this paper outlines a high-performance, high-throughput architecture for high-radix Montgomery Multiplication. We improved the idea of high-radix Montgomery Modular Multiplication algorithm presented in [15] by using a lookup-table approach. This enabled us to efficiently reduce the delay in the Montgomery product-term generation loop, thereby achieving high performance.

3. Montgomery Modular Multiplication

For cryptographic applications, the choice of crypto system combined with the dedicated rather than general-purpose hardware is critical in reducing the energy used. So, one can employ fast multiplication techniques such as Montgomery Modular Multiplication, which is used to calculate the Exponentiation operation; it is fast and suitable for hardware implementation. Montgomery Multiplication replaces the division by implementing the modulus operation with simple right-shifts. This is easier to implement but requires some pre- and post- calculations due to performing transformation to/from Montgomery domain while performing the Exponentiation operation [18].

The application of Montgomery algorithm on two numbers X and Y of n bits gives the following result:

$$C = MM(X, Y) = X * Y * r^{-1} mod\ N$$

Where $r = 2^n$, and n is number of bits with gcd (r, N) = 1. Since $r = 2^n$, it implies that modulus N be an odd integer, where this condition is satisfied for RSA.

The standard Montgomery algorithm with reduction is represented in Algorithm 1.

Algorithm 1: Montgomery Modular Multiplication

Inputs: $X_{n-1:0}$, $Y_{n-1:0}$, $N_{n-1:0}$ with $0 \leq X, Y < N$
Output: $S = X*Y\ (r^{-1}) \bmod N$
n: Number of bits in operands
x_i: i^{th} bit of X, S_0: LSB of S

```
1. begin
2. S = 0
3. for (i = 0; i < n; i++) {
4.     S = S+x_i*Y;
5.     S = S+S_0*N;
6.     S = S div 2;}
7. if (S ≥ N) then S = S−N;
8. return S
9. end
```

4. Proposed Radix-16 Montgomery Modular Multiplication

The proposed Radix-16 Montgomery Modular Multiplication algorithm is presented in Algorithm 2.

Algorithm 2: Radix-16 Montgomery Modular Multiplication

Inputs: $X_{n-1:0}$, $Y_{n-1:0}$, $N_{n-1:0}$ with $0 \leq X, Y < N$
Output: $S = (X*Y\ (r^{-1}) \bmod N)$
n: Number of bits in operands
x_i: i^{th} bit of X, $I_{<3:0>}$ is stored 4-bit value in 16 × 8 lookup table

```
1. begin
2. S = 0
3. for (i = 0; i < n/4; i++) {
4.     O = S+X_<3:0>*Y;
5.     S = O+I_<3:0>*N;
6.     S = S div 16;
7.     X = X div 16;}
8. if (S ≥ N) then S = S−N;
9. return S;
10.end
```

Hardware designed to execute in parallel

The algorithm implementation is presented as flow-chart in Fig. 1.

4.1. High Radix Multiplication

In general, Radix-r versions of multiplication recurrences are mainly in two ways:

a. Multiplication with right-shifts: top-to-bottom accumulation.
b. Multiplication with left-shifts: bottom-to-top accumulation.

Here, we opted for first option as reduction also can be done easily with right-shifts such that we can operate the total design in same bit-length (with penalty of 4 bits for Radix-16 to store MSB's) by shifting the 4 bits to right.

An example is presented below.

Consider X – 0111, Y – 1110 numbers in Montgomery domain with N – 0101, assume 'n' as 4 bits.

Initially S = 0

O = 0 + (0111 * 1110) = 1100010

From the lookup table (stored value), if $O_{<3:0>}$ = 0010 & $N_{<3:0>}$= 0101 then -> $I_{<3:0>}$ = 0110

S = 1100010 + (0101 * 0110)

S = 1000**0000** **<** 4-bits of LSB are zeros **>**

S = S div 16 => S = 00001000

X = X div 16 => X = 0000 – loop ends as 'n' is 4 bits only.

Here S > N, So S = 00001000 – 0101 = 00000011

Final output in Montgomery domain is => 0011

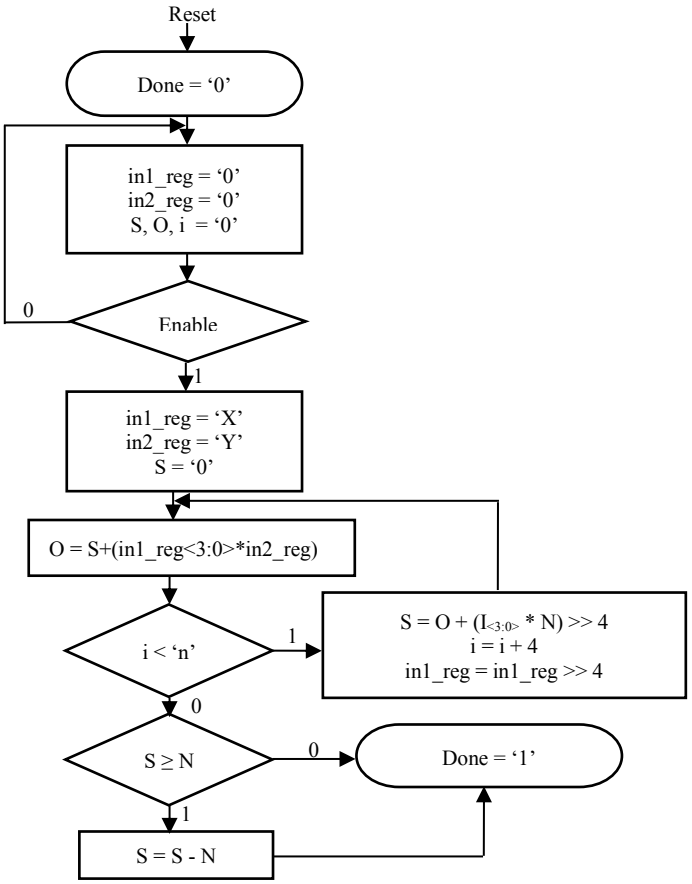


Fig. 1. Radix -16 Montgomery algorithm flowchart

Example shows that it takes "'n/4' + '1' + '1 (if S > N)'" cycles to get the product output. So, for n =1024 bit, the algorithm takes "256+1+1+2 = 260" cycles in the partial product loop. The extra two cycles for inputs loading and getting multiplied result to out. Even though, the intermediate sums are more than 'n' bits (n+4 bits for radix-16), but the final output will be in 'n' bits (always result will be lesser than mod value) only. The design architecture is presented in Fig. 2.

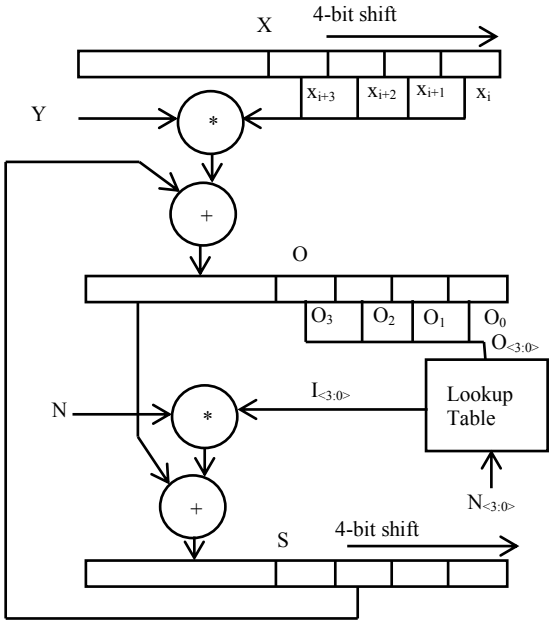


Fig. 2. Radix -16 Montgomery Multiplication architecture

4.2. Lookup-Table based approach

The proposed work highlights efficient hardware architecture to perform high speed Modular Multiplication operation. From Algorithm 2, index bits ($I_{<3:0>}$) depend on LSBs of $O_{<3:0>}$ and LSBs of modulus value $N_{<3:0>}$. The basic reason to have index bits is to make 4 LSBs of output (O) as zeros by adding product of N * $I_{<3:0>}$ to it such that sum (S) can be right-shifted by 4-bits. In modular arithmetic adding/subtracting the modulus or multiple of modulus to/from a number does not affect the number's value [15]. So, these index bits depend on the $O_{<3:0>}$ and $N_{<3:0>}$. Based on $O_{<3:0>}$ bits, N should be multiplied by index bits ($I_{<3:0>}$) and added to O such that the 4 LSBs of S are zeros. O will have 16 combinations of variations and N will have 8 combinations of variations (since N is an odd number). Index is a pre-calculated 4-bit 16 × 8 lookup-table for Radix-16 Montgomery Modular Multiplier. Table 1 shows Index bits ($I_{<3:0>}$) for few of $O_{<3:0>}$ with $N_{<3:0>}$.

Table 1. Index Bit Values for Radix-16 Montgomery Multiplier

$O_{<3:0>}$	$N_{<3:0>}$	Index – $I_{<3:0>}$
0000	*<Any odd value>	0000
0001	0001	1111
	0011	0101
	0101	0011
	0111	1001
	1001	0111
	1011	1101
	1101	1011
	1111	0001
	0001	1110
	0011	1010
	0101	0110

0010	0111	0010
	1001	1110
	1011	1010
	1101	0110
	1111	0010
…….	……..	……..
1111	1111	0001

From Table 1, it is evident that O added with multiples of N with index will make four LSBs of sum as zeros, which is requirement for Radix-16 as per Algorithm 2.

These index bits are calculated manually depending on the LSBs of O and N and stored in a lookup table. The LUT values are obtained after computing Extended Euclidian algorithm. The hardware could incorporate Extended Euclidian algorithm, instead of LUTs, but it would take significant area and time and hence would slow down the Exponentiation operation. Depending on the LSBs (for Radix-16: 4 bits) of O and N, selector shown in Fig. 3, picks the required index value from the lookup table. This will reduce area and will give higher performance in terms of speed.

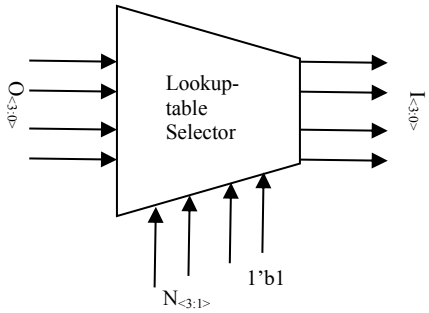


Fig. 3. Selector block

5. Montgomery Modular Exponentiation

The Montgomery Modular Multiplication algorithm is more suited when several Modular Multiplications with respect to the same modulus have to be performed, as required in the case of Modular Exponentiation. By this way, the ratio between the overhead associated with the conversions and the actual modular arithmetic computation is reduced and the Montgomery approach can outperform classical Exponentiation methods.

5.1. Adopting Binary Method for Montgomery Modular Exponentiation Hardware Implementation

The binary method, known as 'square and multiply', is the simplest and easiest method to compute Modular Exponentiation [16]. There are two commonly used Modular Exponentiation algorithms in binary method which can convert the Modular Exponentiation into a sequence of Modular Multiplication operations, i.e., the Right-to-Left (R-L) binary algorithm and the Left-to-Right (L-R) binary algorithm. The binary method scans the bits of the exponent either from left-to-right or from right-to-left. A squaring is performed at each step. Subsequently, depending on the scanned bit value, a subsequent multiplication is performed. Both L-R and R-L binary exponentiation algorithms have the same number of computations for multiplication and squaring operations, therefore they have the same computational complexity.

But there are still few differences that exist between these two algorithms, which are listed below:

a. The scan patterns of these two algorithms are different and squaring operations are executed in different procedures [17].
b. The R-L binary exponentiation method, which scans the exponent bits from the Least Significant Bit and allows squares and multiplications to run in parallel. So, R-L implementation has greater encryption speed than L-R implementation [14].

The ability to generate digit powers often enables R-L exponentiation to be performed significantly faster and in less space than left-to-right methods using the same but restricted space [17]. Thus, the R-L binary exponentiation method has been adopted and implemented for this design.

5.2. Montgomery based R-L Binary Exponentiation

The Montgomery based R-L binary Exponentiation algorithm presented in Algorithm 3.

Algorithm 3: Montgomery based R-L binary exponentiation

Inputs: e, M, N, r^2; e = $(e_{n-1}, e_{n-2}, \ldots e_3, e_2, e_1, e_0)_2$
Output: C= M^e mod N

```
1. C = Montg (1, r^2, N), S = Montg (M, r^2, N);
2. begin
3. for i = 0 to n - 1 do {
4.     if (ei = 1) then C = Montg (C, S, N);
5.     S = Montg (S, S, N);}
6. C = Montg (C, 1, N);
7. return C;
8. end
```

The hardware architecture used to perform RSA Encryption procedure in Montgomery Exponentiation domain is illustrated in Fig. 4. Where Montg() and MM represents Montgomery Modular Multiplier.

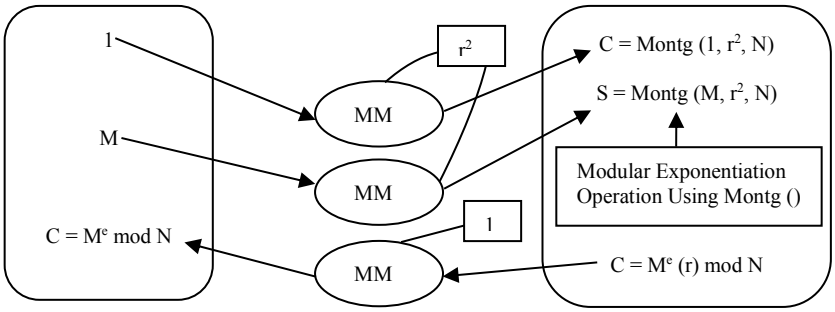


Fig. 4. Encryption block diagram

6. Implementation Results and Analysis

6.1. Implementation

The design was tested on Pico Computing M503 FPGA (device: XC6VLX550T-2ff1759) development board. The design was synthesized using Xilinx ISE 14.7. The design has been simulated up to key-sizes 4096 bits; exponentiation of bit-length 1024 and 2048 were tested and verified on Virtex-6 FPGA. The design has been verified against NIST test-vectors (bit-lengths: 1024 and 2048) [20]. Testing of RSA Encryption/Decryption engine using NIST test-vector framework requires incorporation of SHA-1 hashing of the message. SHA-1 operation was performed using software. The complete Modular Exponentiation design was tested on FPGA (refer Fig. 5), the results of which are shown in Table 2 below. The proposed architecture allows the computation of Modular Exponentiation for keys from 1024 bits to 4096 bits. The computation time for Montgomery Modular Exponentiation with Radix-16 Modular Multiplier for 1024-bit is 6.35 ms. Table 2 shows the computational delays for 1024/2048/4096 encryptions with Radix-2 and Radix-16 Montgomery Modular Multiplier designs.

Table 2. Computation delay for Modular Exponentiation with Radix-2 and Radix-16 Montgomery Modular Multiplier

Computation delay (in ms) for Exponentiation – 1024, 2048 and 4096 Bits								
Reference	[6]		This work		This work		This work	
Size	1024		1024		2048		4096	
Device	Virtex-4		Virtex-6		Virtex-6		Virtex-6	
Radix	2	16	2	16	2	16	2	16
Number of Clock Cycles	NA	NA	1058832	269328	4214800	1062928	16818192	4222992
Delay (in ms)	85.63	NA	21.12	6.35	157.5	43.16	1219.5	320.4

6.2. Results

Theoretically, for 1024-bit Exponentiation using Radix-16 multiplier, computation delay, T_{total} = T × ((260 × 1028) + 2 * 1024) = 6.35 ms; this is the time taken for encryption, where the first 260 represents the number of cycles for Montgomery Multiplication, 1028 (1024 multiplications (as per bit-length) + 4 multiplications for integer-to-Montgomery and Montgomery-to-integer conversions) represents the iterations and 2 * 1024 represents cycles required for pre-calculation (r^2). The critical path delay (T) of the 1024/2048/4096-bit Encryption with Radix-16 multiplier is 23.559ns/41ns/75.87ns, respectively.

The hardware validation test-setup of RSA-encryption/decryption is shown in Fig. 5. A C program was written to load the test-vectors into the RSA engine in FPGA and check the output and log the results.

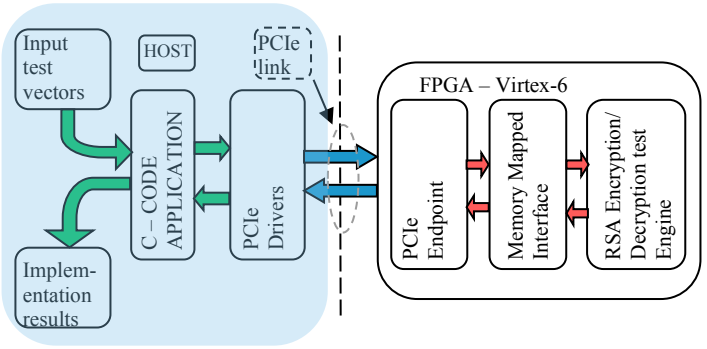


Fig. 5. Hardware validation of RSA Encryption/ Decryption

6.3. Comparison with related work

The results obtained were checked for correctness against NIST results. The implementation synthesis reports for 1024, 2048, 4096 bits are shown in Table 3 and Table 4. From Table 3, it is evident that our 1024-bit Exponentiation implementation using Radix-2 multiplier is occupying 9% Slice LUTs; but in [7], for the same Virtex-6 FPGA device (XC6VLX550T-2ff1759), for 32-bit Exponentiation architecture, the occupied area is 93% Slice LUTs. Our Radix-2 design has extremely low-area as compared to the implementation in [7].

In [6], for Radix-2 1024-bit Montgomery Exponentiation, the Device Utilization has been reported as 94% in Virtex-4 and it takes 85.63ms for 1024-bit Exponentiation; while our Radix-2 1024-bit design reports Device Utilization of 9% for Virtex-6 FPGA and it takes 21.12ms for 1024-bit Exponentiation. Comparison reports are shown in Table 2, Table 3 and Table 5. The Montgomery Exponentiation design in [6, 7] are Radix-2 based Montgomery Multiplication, while our design features Radix-16 Montgomery Multiplier.

To the best of our knowledge, we were unable to find any results for Montgomery Modular Exponentiation with Radix-16 Montgomery Multiplier for comparison with our design.

Table 3. Utilization Summary of Montgomery Exponentiation with Radix-2 Montgomery Modular Multiplier

Utilization						
Reference	[7]	[7]	[6]	This work	This work	This work
Bit Size	16	32	1024	1024	2048	4096
Device	Virtex-6	Virtex-6	Virtex-4	Virtex-6	Virtex-6	Virtex-6
Slice LUTs	243222 (71%)	319631 (93%)	10170 (94%)	32854 (9%)	65473 (19%)	130987 (38%)

Table 4. Utilization Summary of Montgomery Exponentiation with Radix-16 Montgomery Modular Multiplier

Utilization			
Reference	This work	This work	This work
Bit Size	1024	2048	4096
Slice Registers	18548 (2%)	36992 (5%)	73864 (10%)
Slice LUTs	46362 (13%)	92505 (26%)	184775 (53%)

6.4. Performance Analysis

Performance reports are shown in Table 5 and Table 6, which shows the results for the 1024/2048/4096-bit Montgomery Modular Exponentiation using Radix-2 and Radix-16 Montgomery Modular Multipliers. In the tables, bits/sec = (frequency of encryption (or decryption) * bit size)/ (no. of cycles required for the encryption); digital signings/sec = (frequency of encryption (or decryption))/ no. of cycles required for the encryption. For n-bit implementation, the 'r^2' calculation will take 2n clock cycles. Proposed Radix-16 Montgomery modular multiplier takes n/4+4 clock cycles and finally Exponentiation takes $2n+(n+4)(n/4+4) = n^2/4+7n+16$ cycles.

Table 5. Speed obtained for Montgomery Exponentiation with Radix-2 Montgomery Modular Multiplier

Bit Size	Device	Reference	Frequency of Operation	Baud rate (Kbits/sec)	Digital Signings/sec
1024	Virtex-4	[6]	12.29	11.95	11.67
1024	Virtex-6	This work	50.16	48.51	47.37
2048	Virtex-6	This work	26.78	13.01	6.35
4096	Virtex-6	This work	13.85	3.37	0.82

Table 6. Speed obtained for Montgomery Exponentiation with Radix-16 Montgomery Modular Multiplier

Bit Size	Device	Reference	Frequency of Operation	Baud rate (Kbits/sec)	Digital Signings/sec
1024	Virtex-6	This work	42.45	161.4	157.6
2048	Virtex-6	This work	24.39	47.5	23.16
4096	Virtex-6	This work	13.18	12.8	3.12

6.5. Analysis with CSA and FPGA DSP Blocks

We have analyzed our design by implementing Carry Save Adder (CSA) in the Montgomery product generation loop and achieved further 8% improvement in frequency of operation for 1024/2048/4096-bit Montgomery Exponentiation with Radix-2 Montgomery Modular Multiplier.

We have also analyzed the Exponentiation architecture by using Virtex-6's DSP48 blocks; we found a 25% drop in frequency of operation in 1024/2048/4096-bit Montgomery Exponentiation with Radix-16 Montgomery Modular Multiplier. This was due to additional routing delays being introduced in the critical path while utilizing the FPGA DSPs.

7. Conclusion

This paper describes efficient hardware architecture for computing high-radix Montgomery Multiplication which greatly reduces the computation time of Montgomery Modular Exponentiation operation. The proposed architecture supports key-sizes up to 4096 bits using the same hardware architecture. The design has been synthesized and implemented using Verilog HDL with the help of Xilinx ISE 14.7. The design has been simulated (up to key sizes 4096 bits), and tested on Pico Computing M503 FPGA (device: XC6VLX550T-2ff1759) development board (up to key-sizes 2048) and verified against NIST test-vectors (bit-lengths: 1024 and 2048).

The computation delay using our proposed implementation takes 6.35ms, 43.16ms, 320.4ms for computing 1024/2048/4096 bits of Modular Exponentiation, respectively. The performance results for our proposed hardware architecture have been captured and compared in Table 2, Table 5 and Table 6.

The throughput and Digital signings/sec could be further improved by incorporating multiple parallel exponentiation engines in the FPGA.

References

[1] R. Rivest, A. Shamir, L. Adleman, "A method for obtaining digital signatures and public key cryptosystems", Commun. ACM, vol. 21, no. 2, pp. 120-126, Feb. 1978.

[2] N. Nedjah, L.M Mourelle, “Parallel computation of modular exponentiation for fast cryptography”, Int. Journal of. High Performance Systems Architecture, Vol. 1, No. 1, 2007, Copyright © 2007 Inderscience Enterprises Ltd.

[3] E. Öksüzoglu and E. Savas, "Parametric, Secure and Compact Implementation of RSA on FPGA," 2008 International Conference on Reconfigurable Computing and FPGAs, Cancun, 2008, pp. 391-396.

[4] S. Li et al., "Research in Fast Modular Exponentiation Algorithm Based on FPGA," 2019 11th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA), Qiqihar, China, 2019, pp. 79-82.

[5] S. S. Ghoreishi, M. A. Pourmina, H. Bozorgi and M. Dousti, "High Speed RSA Implementation Based on Modified Booth's Technique and Montgomery's Multiplication for FPGA Platform," 2009 Second International Conference on Advances in Circuits, Electronics and Micro-electronics, Sliema, 2009, pp. 86-93.

[6] A. Nadjia, A. Mohamed and I. Mohamed, "Montgomery modular exponentiation on FPGA," 2012 24th International Conference on Microelectronics (ICM), Algiers, 2012, pp. 1-4.

[7] Manish Bansal, Amit Kumar, Aakanksha Devrari, and Abhinav Bhat, “Implementation of Modular Exponentiation using Montgomery Algorithms”, International Journal of Scientific & Engineering Research (IJSER), Volume 6, Issue 11, November-2015.

[8] Xiaodong Yan and Shuguo Li, "Montgomery multiplier based on secondary booth encoded algorithm," 2007 7th International Conference on ASIC, Guilin, 2007, pp. 197-200.

[9] S. Wang, W. Lin, J. Ye and M. Shieh, "Fast scalable radix-4 Montgomery modular multiplier," 2012 IEEE International Symposium on Circuits and Systems (ISCAS), Seoul, 2012, pp. 3049-3052.

[10] G. Sassaw, C. J. Jiménez and M. Valencia, "High radix implementation of Montgomery multipliers with CSA," 2010 International Conference on Microelectronics, Cairo, 2010, pp. 315-318.

[11] P. Amberg, N. Pinckney and D. M. Harris, "Parallel high-radix Montgomery multipliers," 2008 42nd Asilomar Conference on Signals, Systems and Computers, Pacific Grove, CA, 2008, pp. 772-776.

[12] V. Bunimov, M. Schimmler, B. Tolg, “A complexity-effective version of Montgomery’s algorithm,” in Workshop on Complexity Effective Designs, ISCA’02, Anchorage, Alaska, May 2002.

[13] S. Prasanth kumar, K.J.Jegadish kumar and B.Partibane, “Efficient Modular Exponentiation Architectures for RSA Algorithm”, International Journal of Engineering Research in Electronic and Communication Engineering (IJERECE), Vol 3, Issue 5, May 2016.

[14] N. Pinckney and D. M. Harris, “Parallelized radix-4 scalable Montgomery multipliers,” Journal of Integrated Circuits and Systems, vol. 3, no. 1, pp. 39–45, Mar. 2008.

[15] M. Knezevic, F. Vercauteren and I. Verbauwhede, "Faster Interleaved Modular Multiplication Based on Barrett and Montgomery Reduction Methods," in IEEE Transactions on Computers, vol. 59, no. 12, pp. 1715-1721, Dec. 2010.

[16] M. Shieh, J. Chen, H. Wu and W. Lin, "A New Modular Exponentiation Architecture for Efficient Design of RSA Cryptosystem," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 16, no. 9, pp. 1151-1161, Sept. 2008.

[17] Colin D. Walter "Right-to-Left or Left-to-Right Exponentiation?" First International Workshop on Constructive Side-Channel Analysis and Secure Design COSADE 2010, pp. 40-46.

[18] P. L. Montgomery, “Modular Multiplication without Trial Division,” Math. Computation, Vol. 44, no. 170, pp. 519-521, 1985.

[19] S. Vollala, B. S. Begum, A. D. Joshi and N. Ramasubramanian, "High-radix Modular Exponentiation for hardware implementation of Public-Key Cryptography," 2016 International Conference on Computing, Analytics and Security Trends (CAST), Pune, 2016, pp. 346-350.

[20] https://csrc.nist.gov/Projects/Cryptographic-Algorithm-Validation-Program/Digital-Signatures#rsavs